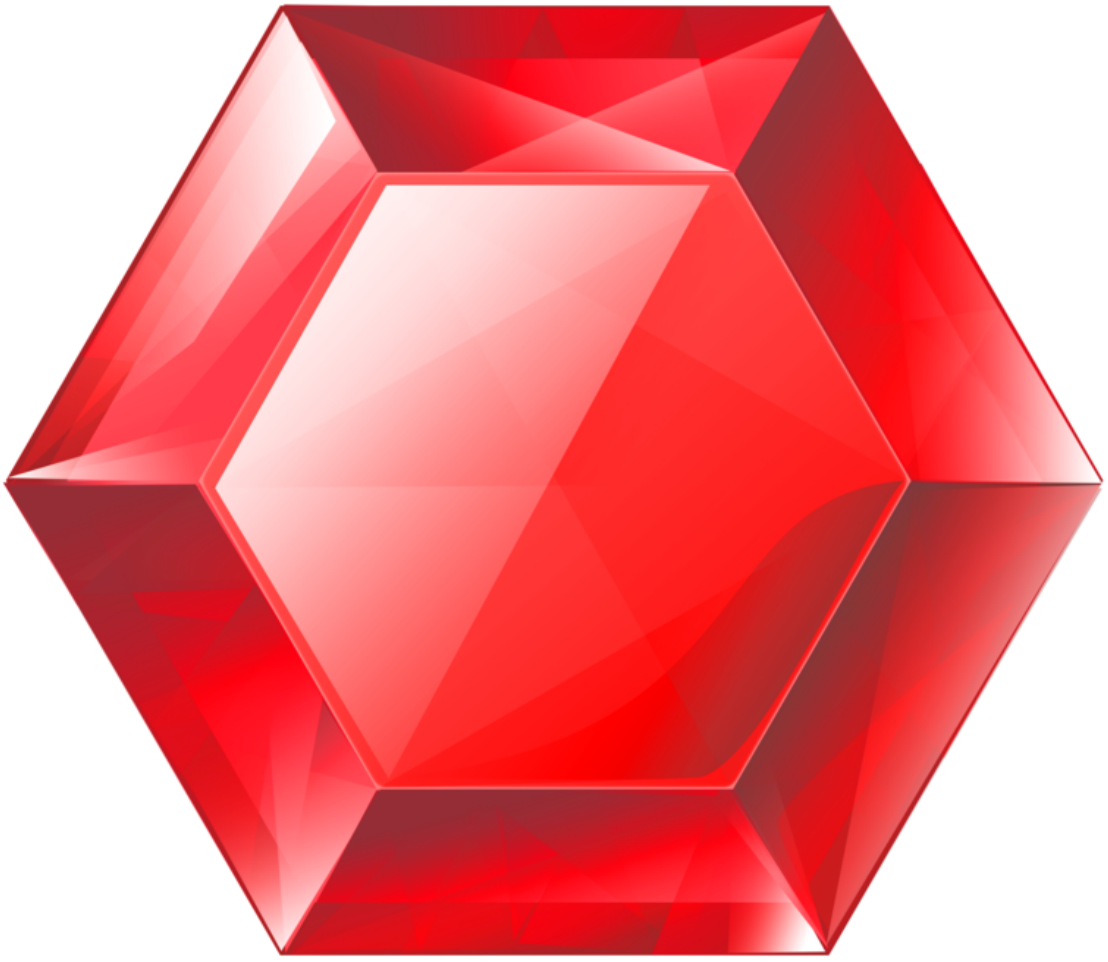
gemstone-py vs gemstone-rs

Install paths, use cases, maturity, and feature matrix



Generated from the Markdown sources in gemstone-rs/docs.

Contents

gemstone-py vs gemstone-rs

`gemstone-py` and `gemstone-rs` solve related problems at different maturity levels.

Use `gemstone-py` when you want the most complete Python experience today: Python sessions, async examples, web framework adapters, native acceleration, codegen, VS Code workbench integration, and a Python database explorer.

Use `gemstone-rs` when you want Rust services, CLIs, workers, or tooling to talk to GemStone/S directly without keeping Python in the process.

Install Paths

Goal	gemstone-py	gemstone-rs
Application library	`python -m pip install gemstone-py`	`cargo add gemstone-rs`
Native acceleration	`python -m pip install "gemstone-py[fast]"`	Built into the Rust GCI path once `libgcirpc` is available
Examples from source	`python -m pip install -e ".[examples]"`	`cargo run -p gemstone-rs --example quickstart`
Example discovery/run	`gemstone-examples hello`, `list`, `plan3-map`, and launch commands	`gemstone-rs hello`, `compare gemstone-py`, `examples list`, `map`, `show`, and source-checkout `run`
CLI tooling	Python modules and examples	`cargo install gemstone-rs-cli`
Local explorer	`python-gemstone-database-explorer`	`cargo install gemstone-rs-explorer`
VS Code	`gemstone-py Workbench`	`gemstone-rs Workbench`

Both projects use the same GemStone-style environment:

```
export GS_LIB=/opt/gemstone/product/lib
export GS_STONE=gs64stone
export GS_USERNAME=DataCurator
export GS_PASSWORD=your-password
```

`GS_STONE_NAME` is accepted as a stone-name alias where the clients support it. Use `GS_LIB_PATH` when you want to point directly at a specific `libgcirpc` file.

Best Fit

Use case	Better choice	Why
Python web app	gemstone-py	FastAPI and Litestar examples are already first-class.
Python scripts and notebooks	gemstone-py	Lower friction for Python teams.
Rust service or worker	gemstone-rs	Native Rust API, Rust errors, Rust ownership, Cargo distribution.
Rust CLI tooling	gemstone-rs	CLI and explorer can run without Python.
Example-driven onboarding	gemstone-py today, gemstone-rs catching up	gemstone-py has broader runnable examples; gemstone-rs now has no-live hello, a direct comparison command, an installed CLI example index, feature map, and source-checkout runner.
VS Code database/codegen workflow	gemstone-py today, gemstone-rs later	gemstone-py has the more mature explorer; gemstone-rs has the cleaner Rust backend direction.
Shared low-level bridge	gemstone-rs	The Rust API is the right place to isolate GCI safety and ownership.

Maturity Matrix

Capability	gemstone-py	gemstone-rs
Stable install path	Mature: PyPI and TestPyPI	Early: crates.io workflow and verification added
Native bridge	PyO3 extension path	Rust GCI crate and safe API
Sync API	Mature	Initial safe API
Async API	Mature enough for examples and tests	Not yet a core feature

Capability	gemstone-py	gemstone-rs
Web frameworks	FastAPI, Litestar, Django examples	Axum service sketch added; full app example still planned
Codegen	Python wrapper workflow	Rust wrapper workflow with preview/diff/check/generate
Browser API	Used by database explorer	CLI/explorer API for dictionaries/classes/methods/source
Local explorer	More mature Python app	Minimal Rust explorer proving the API
VS Code extension	More complete product flow	Thin command/workbench layer over Rust CLI and explorer
Live tests	Broader coverage	Growing smoke suite for login/eval/perform/globals/transactions/codegen
Release automation	More complete	Added crates/VSIX/GitHub verification path
Docs	Broader and polished	Catching up with guide, cookbook, article, comparison, example index, PDFs

Feature Matrix

Feature	gemstone-py status	gemstone-rs status
Login/logout	Supported	Supported
`3 + 4 == 7` smoke	Supported	Supported
Global put/get	Supported	Supported
String round-trip	Supported	Supported
`perform` and `perform_oop`	Supported	Supported
Commit/abort	Supported	Supported
OOP handles/export set	Supported	Supported
	Supported	Supported through `Browser`, CLI, and explorer

Feature	gemstone-py status	gemstone-rs status
Browser dictionaries/ classes/protocols/ methods/source		
Generated wrappers	Supported	Supported for selected classes/methods
Codegen diff/check/ generate	Supported	Supported
Live codegen discovery	Supported	Supported through CLI/API
FastAPI demo	Supported	Not applicable
Litestar demo	Supported	Not applicable
Axum/Actix demo	Not applicable	Axum sketch documented; full app example still planned
Installed examples index	`gemstone-examples hello`, `list`, `plan3-map`	`gemstone-rs hello`, `compare gemstone-py`, `examples list`, `map`, `show`, `run --dry-run`, plus JSON for tooling
Database explorer	Mature Python app	Rust explorer has browser UI and local API; still less polished
VS Code sidebar workflow	More complete	Rust workbench has sidebar commands and embedded explorer webview

Current Gap Analysis

`gemstone-py` remains better when the user wants a batteries-included Python product surface: package extras, web adapters, async examples, a mature database explorer, and an examples launcher aimed at Python users and installed environments.

The most useful gemstone-rs catch-up work is to keep copying the product shape, not the implementation language:

- Installed discoverability: `gemstone-rs examples list` now mirrors the gemstone-py example map and gives VS Code structured JSON.
- No-live sanity check: `gemstone-rs hello` mirrors `gemstone-examples hello` so users can verify the CLI before configuring GemStone.
- Direct comparison: `gemstone-rs compare gemstone-py` turns this guide into a

compact CLI summary with JSON output for tooling.

- Feature stream map: `gemstone-rs examples map` mirrors `gemstone-examples plan3-map`, but frames each stream in Rust crates, examples, docs, and gemstone-py reference points.
- Source-checkout launching: `gemstone-rs examples run <name>` now executes the matching Cargo example, with `--dry-run` for CI and documentation checks.
- Editor workflow: `GemStone RS: Show Example Commands` exposes the same map in the Rust workbench and can run selected Cargo examples in a terminal.
- Remaining gap: gemstone-rs still needs fuller Rust web examples, richer explorer screens, and more generated wrapper demos before its onboarding feels as complete as gemstone-py.

How They Should Work Together

The best long-term shape is not competition between the projects. It is a clean boundary:

- `gemstone-rs` owns the direct Rust/GCI interface and safe Rust API.
- `gemstone-py-native` can become a thin PyO3 layer over the Rust core.
- `gemstone-py` remains the Python API, examples, and Python web integration.
- The Python and Rust explorers can share concepts and eventually converge on common codegen workflows.

That gives Python users the friendliest path and Rust users a direct path, while keeping unsafe GCI behavior isolated in one Rust layer.

Recommendation

For production Python projects, start with `gemstone-py`.

For Rust-native services, CLIs, workers, and tooling, start with `gemstone-rs`, but treat it as an early API that should be validated against a live stone in your environment.

For VS Code and visual codegen work, use the existing Python explorer as the product reference and keep moving `gemstone-rs-explorer` toward the same level of capability.

This PDF was generated locally from `docs/`. If you edit the Markdown sources, rerun `python docs/build_pdf_docs.py`.